

Network-Controlled Custom CMD Panel Using TCP Sockets

Course: Computer Networks

Project Type: Semester Project

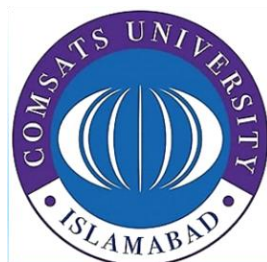
Submitted By:

- **Hamza Ali (124-BSE-3B-FALL24)**
- **Akif Naveed (129-BSE-3B-FALL24)**

Submitted To:

- **Dr. Mushtaq Khan**

COMSATS UNIVERSITY ISLAMABAD WAH CAMPUS
--



Abstract

This project presents the design and implementation of a **Network-Controlled Custom Command Line Panel** using **TCP socket programming in Python**. The system follows a client–server architecture where a graphical client application communicates with a server over a TCP connection to execute predefined custom commands. The project demonstrates fundamental computer networking concepts such as TCP communication, socket binding, message framing, command parsing, and multi-threaded client handling. The system is developed for educational purposes to simulate a customized command-line interface like CMD or Bash while ensuring controlled and secure command execution.

1. Introduction

Computer networks play a vital role in modern distributed systems. Understanding how devices communicate over a network is essential for computer science students. This project focuses on implementing a **custom TCP-based command execution system** that allows a client to send commands to a remote server and receive structured responses in real time.

Unlike traditional operating system command-line interfaces, this system uses **user-defined commands** with simulated outputs, making it safe, educational, and flexible. The project provides hands-on exposure to client–server communication, socket programming, and GUI-based networking applications.

2. Objectives

The main objectives of this project are:

- To demonstrate real-time communication between two machines using **TCP sockets**.
- To design a **custom command-line protocol** for client–server interaction.
- To implement a **graphical client interface** using Python Tkinter.
- To simulate system, network, and utility commands in a controlled environment.
- To strengthen understanding of **client–server architecture** and networking fundamentals.

3. Scope of the Project

This project is limited to:

- Text-based command transmission over TCP.
- Predefined and simulated command execution.
- Single-server, multiple-client support using threading.
- Local network communication (same Wi-Fi or hotspot).

The system does not execute real operating system commands for security reasons.

4. System Architecture

The system follows a **client-server architecture**:

1. The **Server Application** runs on one machine and listens on a specific TCP port.
2. The **Client GUI Application** runs on another machine and connects using IP address and port number.
3. The client sends commands as structured JSON messages.
4. The server parses commands, executes predefined handlers, and sends responses.
5. The client displays responses in a custom command-line styled GUI.

5. Technologies Used

- **Programming Language:** Python 3.x
- **Networking:** TCP Socket Programming
- **GUI:** Tkinter (Python)
- **Data Serialization:** JSON
- **Concurrency:** Multi-threading
- **Optional Library:** pip install psutil (for system information)

6. Communication Workflow

1. Both client and server machines are connected to the same network.
2. The server binds to an IP address and port and starts listening.
3. The client connects to the server using the server's IP and port.

4. A TCP connection is established.
5. Commands are sent from client to server as JSON messages.
6. The server processes commands and sends structured responses.
7. The client displays output in real time.

7. Features of the System

- Custom command-line interface
- Real-time TCP communication
- Graphical client panel
- Expandable command architecture
- Simulated system and network commands
- Command logging functionality
- Multi-client handling using threads

8. Implemented Commands

8.1 System Commands

- **TIME** – Returns server date and time

8.2 Network Commands

- **PING** – Simulated ping response

8.3 Utility and Fun Commands

- **ECHO** – Returns input text
- **JOKE** – Displays a programming joke
- **QUOTE** – Displays a motivational quote
- **RANDOM_FACT** – Shows a random networking or programming fact
- **HELP** – Lists all available commands
- **SYS_LOGS** – Displays recent command logs

9. Security Considerations

- No real OS commands are executed.
- Input validation is applied for calculation commands.

- Communication is limited to predefined command handlers.
- Server controls all command execution logic.

10. Limitations

- Works only on local networks.
- No encryption implemented.
- Limited to text-based responses.
- File transfer is read-only and limited.

11. Future Enhancements

- Implement SSL/TLS encryption.
- Add authentication and user roles.
- Enable real-time file transfer.
- Improve GUI with command history and auto complete.
- Add remote administration features.

12. Conclusion

The **Network-Controlled Custom CMD Panel Using TCP Sockets** successfully demonstrates core computer networking concepts through a practical implementation. The project provides valuable experience in socket programming, client-server communication, GUI development, and command parsing. It serves as an effective educational tool for understanding TCP-based networking systems and can be extended further for advanced networking applications.

13. References

- Python Official Documentation
- Computer Networking Concepts (TCP/IP Model)
- Tkinter GUI Documentation

End of Report
